

Project Viva Study Guide

Multi-Agent Infrastructure Compliance Engine

Your situation: You built (or submitted) this project, and now you have to explain every part of it to a professor with 30 years of experience. This guide walks you through the project the way you would actually explain it — starting from the big idea, then the parts, then the code lines that matter, then likely questions.

PART 1 — The 60-Second Pitch (memorize this!)

When the professor says "Tell me about your project", this is what you say, calmly:

"Sir/Madam, my project is an automated auditor for large infrastructure projects like highways and bridges in Uttarakhand. When a government department prepares a 'Detailed Project Report' or DPR, it usually goes to the Forest Department, the Engineering Department, the Mining Department and so on. Each one finds problems and sends it back. My system uses four AI agents — each one is a digital specialist for one department — and they read the DPR together, search government websites, and produce a report saying exactly which corrections are needed before submission. The whole thing runs on OpenAI's GPT-4o model, orchestrated by a library called LangGraph, and presented through a Streamlit dashboard with live updates and a bilingual (English-Hindi) report download."

That single paragraph contains every keyword the professor wants to hear: **multi-agent**, **DPR**, **LangGraph**, **GPT-4o**, **Streamlit**, **bilingual**.

PART 2 — The Concepts You MUST Understand

If you can explain these six ideas confidently in your own words, you have already won 70% of the viva. Read them slowly.

2.1 What is an "AI Agent"?

A normal chatbot just talks. An **agent** is a chatbot that can **use tools**. Think of the difference like this:

- A *chatbot* is like a person locked in a room with no phone — they can only talk based on what they remember.

- An *agent* is like the same person, but now they have a phone, Google, and a printer. When you ask a question, they decide: *"Should I just answer, or should I first search Google, then look at a PDF, then answer?"*

In your project, each of the four agents has its own set of tools (search functions, PDF readers). The agent looks at the user's request and **decides which tool to call**.

2.2 What is ReAct (Reason + Act)?

This is the pattern your agents use. The agent loops through three steps:

1. **Thought** — *"To answer this question about forest land, I need to search the EDS database."*
2. **Action** — Calls the `search_eds_history` tool with a specific query.
3. **Observation** — Reads the search results.

Then it loops back to Thought again until it has enough information to answer. This is implemented by `create_react_agent` from LangGraph (you'll see this line in every agent file).

2.3 What is RAG (Retrieval-Augmented Generation)?

GPT-4o doesn't know what's in your DPR PDF, and it doesn't know about Uttarakhand PWD circulars from last week. RAG means: **first retrieve the relevant facts, then generate the answer using those facts**.

In your project:

- For Agent 1, the "retrieval" comes from three sources: the user form, the PDF content, and the JSON knowledge base of forest rules. The "generation" is GPT-4o writing the audit report using those three sources.
- For Agents 2, 3, 4, the retrieval is live web search via SerpAPI and Tavily.

2.4 What is LangChain vs LangGraph?

These are Python libraries that make it easier to build agent applications.

- **LangChain** gives you the building blocks (LLM connectors, tools, search wrappers).
- **LangGraph** is the newer piece that handles the **reasoning loop** — it manages the back-and-forth between the LLM and its tools.

When the professor asks *"Why LangGraph and not just plain Python?"*, the answer is: *"LangGraph handles the state management automatically when the agent makes multiple tool calls in sequence. Writing that loop manually means tracking message history, tool call IDs, and recursion limits ourselves — LangGraph does it correctly out of the box."*

2.5 Why two different search APIs (SerpAPI + Tavily)?

The professor will ask this. Here is the honest, technical answer:

- **SerpAPI** returns Google search results — titles, URLs, snippets. Good for **discovery** (*"What tenders exist in this district?"*).
- **Tavily** is AI-optimized — it fetches the actual page content and returns clean extracted text. Good for **reading** (*"What does this tender document actually say?"*).

Using both gives **redundancy** (if one is rate-limited, the other works) and **complementary strengths** (Google for breadth, Tavily for depth).

2.6 Why GPT-4o for Agent 1, GPT-4o-mini for Agents 2–4?

Look closely — Agent 1 uses `model="gpt-4o"` but Agents 2, 3, 4 use `model="gpt-4o-mini"`. This is a **deliberate cost/performance trade-off**:

- Agent 1 has to do a **3-way comparison** across user data, a 90-page PDF, and the JSON knowledge base. That requires the bigger model's reasoning power.
- Agents 2–4 mostly summarize web search results — `gpt-4o-mini` is **cheaper, faster, and has higher tokens-per-minute (TPM) limits**, which matters when each agent makes many search calls.

PART 3 — File-by-File Walkthrough

I'm going through every file. For each one, I'll tell you (a) what it does in one line, (b) the key parts of the code, and (c) what to say when asked about it.



`requirements.txt`

One-line: *The shopping list of Python libraries the project needs.*

Library	Why it's there
<code>langchain</code> , <code>langchain-openai</code> , <code>langchain-community</code> , <code>langchain-core</code>	The framework for building LLM applications
<code>langgraph</code>	Handles the agent reasoning loop (ReAct pattern)
<code>langchain-google-genai</code>	Optional Google Gemini support (not actively used, kept for fallback)
<code>tavily-python</code> , <code>google-search-results</code>	The two search APIs (Tavily and SerpAPI)
<code>duckduckgo-search</code>	A free fallback search if paid APIs fail
<code>python-dotenv</code>	Reads API keys from the <code>.env</code> file safely
<code>pypdf</code> , <code>pymupdf</code> (Fitz), <code>fpdf</code>	PDF reading and writing
<code>pdf2image</code> , <code>pytesseract</code>	Converting PDF pages to images, then running OCR on scanned PDFs
<code>requests</code>	Plain HTTP — used to download EDS PDFs from government websites
<code>streamlit</code>	The web dashboard framework

What to say: *"This file lets anyone reproduce my environment with one command: `pip install -r requirements.txt`. It includes PDF processing libraries because real DPRs are often scanned, search APIs because the agents need live information, and Streamlit for the dashboard."*

`_env` (or `.env`)

One-line: Stores secret API keys outside the code so they never appear on GitHub.

```
OPENAI_API_KEY="..."
SERPAPI_API_KEY="..."
TAVILY_API_KEY="..."
```

Why this matters: Hard-coding API keys inside Python files is a major security mistake. The `.env` file is loaded at runtime via `load_dotenv()` and added to `.gitignore` so it's never committed to a repository.

What to say: *"I separated credentials from code using python-dotenv. This is a standard security practice — if I accidentally pushed my code to GitHub, my paid API keys would still be private."*

`check_api_keys.py`

One-line: A diagnostic script that tests whether all five API keys are alive before you waste hours running the main app.

Key lines to highlight:

```
env_path = r'C:\Users\Aditya\Downloads\Multi agent.env'
env_vars = {}
with open(env_path, 'r') as f:
    for line in f:
        ...
```

This **manually parses** the `.env` file instead of trusting `python-dotenv` — useful when caching causes stale keys.

```
os.environ.update(env_vars)
```

This forces the freshly-read values into the OS environment, bypassing any cached values. The script then calls each API with a tiny test message and prints `[OK]`, `[FAIL]`, or `[WARN]` for each.

What to say: *"Before debugging the agents themselves, I needed to know which keys were the actual problem. This script does a 5-API health check in one go — Gemini, OpenAI, Tavily, SerpAPI, and LangSmith."*

`test_stream.py`

One-line: A minimal sanity check that LangGraph's `stream()` method is actually returning chunks the way I expect.

```
for chunk in agent.stream({"messages": [("human", "search for apples")]}, stream_mode="updates"):
    print("CHUNK:", chunk)
```

Why it exists: When you build live UI telemetry (showing the agent's thoughts in real time in Streamlit), you have to know the exact structure of each streamed chunk. This file is your scratchpad to inspect that structure before wiring it into the main app.

What to say: *"This was my debugging script while I was building the live telemetry feature in Streamlit. I needed to verify what `stream_mode='updates'` actually emits before parsing it in production code."*

`Run_Application.bat`

One-line: A double-click launcher for Windows users so they don't have to open a terminal.

```
@echo off
cd /d "%~dp0"
streamlit run app.py
```

`%~dp0` means "the folder this .bat file is in" — so the user can move the project folder anywhere and the launcher still works.

What to say: *"This is a quality-of-life addition for non-technical users — they double-click and the dashboard opens in their browser."*

`agent1_forest_compliance.py` — The Most Complex File

This is the agent the professor will probe hardest, because it has **three sub-systems**: PDF ingestion, knowledge-base loading, and the 3-way comparison reasoning. Let me walk you through it.

The class structure

```
class ForestComplianceSystem:
    def __init__(self, kb_dir="knowledge_base"):
        self.kb_dir = kb_dir
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0.2, streaming=True)
        self.agent = self._initialize_agent()
```

- `temperature=0.2` → low randomness. We want consistent, factual audits, not creative writing.
- `streaming=True` → enables the live "thinking" feed in Streamlit.

The PDF ingestion pipeline (3 fallback levels)

This is the most impressive engineering in the whole project. Real-world DPRs come in three forms, so the code tries three methods:

```

def ingest_proposal(self, pdf_path):
    # LEVEL 1 - Native text extraction (fast)
    doc = fitz.open(pdf_path)
    text = ""
    for page in doc:
        text += page.get_text() + "\n"

    # LEVEL 2 - If almost no text was extracted, it's a scanned PDF → OCR
    if len(text.strip()) < 50 * total_pages:
        images = convert_from_path(pdf_path)
        for i, img in enumerate(images):
            ocr_text = pytesseract.image_to_string(img)
            text += ocr_text + "\n"

    # LEVEL 3 - If Tesseract isn't installed, fall back to GPT-4o Vision
    except Exception as ocr_e:
        for i, page in enumerate(doc):
            pix = page.get_pixmap()
            img_bytes = pix.tobytes("png")
            b64_img = base64.b64encode(img_bytes).decode('utf-8')
            vision_resp = self.llm.invoke([
                {"role": "user", "content": [
                    {"type": "text", "text": "Extract all text..."},
                    {"type": "image_url", "image_url": {"url": f"data:image/png;base64,{b64_img}"}}
                ]}
            ])

```

What to say (this is your money slide): *"DPR PDFs come in three forms — born-digital PDFs, scanned PDFs, and password-protected scans. I built a three-tier fallback: PyMuPDF first for speed; if that yields too little text I assume it's scanned and run Tesseract OCR; and if Tesseract isn't installed on the host machine, I encode each page as Base64 PNG and send it to GPT-4o's vision endpoint as a serverless OCR. So even with zero pre-installed binaries, the system still works."*

The professor will be impressed by this because it shows you thought about **production reliability**, not just the happy path.

The Smart Two-Pass Summarizer

A 90-page DPR has roughly 200,000+ characters — far beyond GPT-4o's input limit. So `smart_summarizer_dpr` chunks it:

```

chunk_size = 10000

chunks = [full_text[i:i+chunk_size] for i in range(0, len(full_text), chunk_size)]

for idx, chunk in enumerate(chunks):
    # PASS 1: Pull general project metadata
    resp1 = self.llm.invoke([HumanMessage(content="...Extract project name, district,
cost...")])

    # PASS 2: Pull forest-specific data only
    resp2 = self.llm.invoke([HumanMessage(content="...Extract forest area, NPV, KML m
entions...")])

```

What to say: *"I split the document into 10,000-character chunks. For each chunk, I run two passes — one extracts general project metadata, the other extracts only forest-compliance data. This way, even if forest details are buried on page 73, they get extracted instead of getting lost in truncation."*

The Tools Agent 1 has

```

@tool
def search_eds_history(query: str) -> str:
    """Searches forestsclearance.nic.in for historical EDS letters"""
    search_term = f'site:forestsclearance.nic.in OR site:parivesh.nic.in "Essential D
etails Sought" Uttarakhand {query}'
    ...

@tool
def read_eds_pdf_from_url(url: str) -> str:
    """Downloads an EDS PDF and extracts its text"""
    response = requests.get(url, headers=headers, timeout=20)
    pdf_file = io.BytesIO(response.content)
    reader = PdfReader(pdf_file)
    ...

```

Notice the **Google dorking** trick: `site:forestsclearance.nic.in OR site:parivesh.nic.in "Essential Details Sought"` — this restricts Google to just government domains and exact-phrase matching, dramatically improving relevance.

The 3-Way Comparison Prompt

The function `process_dpr` builds three data sources, then feeds them all to the agent in one structured prompt:

- **Source A** — User form data (name, district, cost)
- **Source B** — Extracted DPR content

- **Source C** — Knowledge base JSON rules

Then the prompt explicitly says:

- Compare A vs B → flag  DATA MISMATCH
- Compare B vs C → flag  COMPLIANCE GAP
- Compare A vs C → flag  RISK FLAG

What to say: *"The forensic 3-way comparison is the heart of this agent. By forcing the LLM to label each finding with one of three specific tags, the output is structured enough for downstream parsing while remaining human-readable."*

 `agent2_utility_intelligence.py`

Goal: Find underground utilities (water pipes, fiber cables, gas lines) that the excavation might damage.

This agent is simpler than Agent 1 — it only does live web search, no PDF parsing. Three tools:

```
@tool
def search_government_tenders_serp(query): ...      # SerpAPI → Google results
@tool
def search_utility_data_tavily(query): ...           # Tavily → full page content
@tool
def search_private_infrastructure_serp(query): ...   # SerpAPI → Reliance/Jio/GAIL
```

The prompt then walks the agent through a **5-step plan** (Steps A through D), telling it exactly which tool to use for each step. This is **prompt-engineered scaffolding** — the agent doesn't have to guess the order of operations.

What to say: *"For Agent 2 I dropped to gpt-4o-mini because the work is mostly summarizing search snippets, not deep reasoning. The dual SerpAPI + Tavily setup lets the agent both discover tenders via Google and read the actual tender PDFs via Tavily's content extraction."*

 `agent3_socio_economic_intelligence.py`

Goal: Sniff out local protests, "mafia premium" issues, and find legally registered RBM (River Bed Material) suppliers.

The trick here is **vernacular news scraping**. Look at this query:

```
"site:amarujala.com {district} sadak nirman virodh OR thekedar OR RBM"
```

Those Hindi words mean: *"road construction", "protest", "contractor"*. By searching for Hindi keywords on Hindi news sites, the agent finds local-language news that the typical English-only audit would miss.

What to say: "Agent 3's USP is vernacular-language scraping. A purely English search misses 80% of district-level news. I encoded specific Hindi keywords like 'virodh', 'dharna', 'andolan' so the agent can scan Amar Ujala and Dainik Jagran for the exact signals that matter."

 `agent4_regulatory_compliance.py`

Goal: Resolve conflicts between IS codes, IRC codes, MoRTH specs, and Uttarakhand High Court orders.

Five tools, each pointed at a specific authority:

1. `search_technical_codes_tavily` → BIS (Bureau of Indian Standards) and IRC
2. `search_code_amendments_serp` → Google for latest revisions
3. `search_high_court_orders_tavily` → Uttarakhand HC judgments
4. `search_pwd_circulars_serp` → State PWD circulars
5. `search_morth_specs_tavily` → MoRTH and NHAH specs

What to say: "The 'revision gap' is a real problem — engineering teams often cite IS 456:2000 when the latest amendment is from 2022. Agent 4 cross-checks the design codes in the DPR against the current versions on bis.gov.in and irc.nic.in, plus any state-specific PWD circulars and High Court mining bans that might override central codes."

 `main_orchestrator.py`

One-line: A terminal-based runner that calls all four agents in sequence with a human-approval gate between each.

The key function:

```
def human_in_the_loop(agent_name):
    while True:
        action = input("Action required (approve [a] / reject with feedback [r] / abort [x]): ")
        if action in ['a', 'approve']:
            return True
        elif action in ['r', 'reject']:
            feedback = input(f"Provide feedback for {agent_name}: ")
            ...
```

This is the **"human-in-the-loop"** pattern. After every agent finishes, the user must type `a`, `r`, or `x` before the next agent runs.

What to say: "Compliance audits cannot be fully autonomous — a wrong audit could halt a crore-rupee project. So between every agent I added a manual checkpoint. The user reviews Agent 1's output, approves it, and only then does Agent 2 run."

`app.py` — The Streamlit Dashboard

This is the file the professor will see running on screen. Walk through it section by section.

Cached agent loading

```
@st.cache_resource
def load_agent1():
    return ForestComplianceSystem(kb_dir=...)
```

`@st.cache_resource` means the agent is **initialized once per session**. Without this, every button click would re-create the agent — wasting time and tokens.

What to say: "Streamlit reruns the entire script on every interaction. The `cache_resource` decorator ensures the agents are instantiated only once — otherwise every click would re-load the model and waste OpenAI credits."

Session state

```
if "reports" not in st.session_state:
    st.session_state.reports = None
```

`st.session_state` is Streamlit's way to remember things across reruns. Without it, your agent reports would disappear the moment you click any button.

Live telemetry — the streaming loop

This is in every agent file (`process_dpr`, `process_project`, etc.):

```
for chunk in self.agent.stream(messages, config, stream_mode="updates"):
    for node, values in chunk.items():
        msg = values.get("messages", [None])[-1]
        if node == "agent":
            if hasattr(msg, "tool_calls") and msg.tool_calls:
                for tc in msg.tool_calls:
                    st_callback.markdown(f"**🔗 Using Tool:** `{tc['name']}`")
            if hasattr(msg, "content") and msg.content:
                st_callback.markdown(f"**🗨️ Agent:** {msg.content}")
        elif node == "tools":
            preview = str(msg.content)[:300]
            st_callback.success(f"**✅ Tool Result:** {preview}...")
```

What this does (line by line):

- `self.agent.stream(...)` returns chunks as the agent thinks.

- Each chunk has a `node` — either `"agent"` (the LLM is reasoning) or `"tools"` (a tool just finished).
- If the agent decided to call a tool, we show the tool name.
- If the agent wrote text, we display it.
- When the tool returns, we show a 300-character preview of the result.

This is what produces the live "thinking" you see in the UI.

The Bilingual Translation Module

```
def translate_to_hindi(text):  
    llm = ChatOpenAI(model="gpt-4o", temperature=0.3)  
    prompt = f"Translate the following technical report into professional Hindi. Main  
    tain the exact markdown formatting..."  
    response = llm.invoke([HumanMessage(content=prompt)])  
    return response.content
```

Notice the prompt insists on preserving markdown formatting — otherwise headers and bullets would get lost in translation.

The Base64 HTML Trick

```
def create_html(en_md, hi_md, title):  
    en_b64 = base64.b64encode(en_md.encode('utf-8')).decode('utf-8')  
    hi_b64 = base64.b64encode(hi_md.encode('utf-8')).decode('utf-8')  
  
    html = f"""...  
    document.getElementById('content-en').innerHTML = marked.parse(b64DecodeUnicode("  
{en_b64}"));  
    ..."""
```

Why Base64? When you embed raw Hindi text directly in HTML inside Python f-strings, special characters (quotes, backticks, Hindi unicode escapes) cause syntax errors and broken pages. Base64 encoding lets you carry **arbitrary text safely** through the HTML and decode it cleanly in the browser using `b64DecodeUnicode`.

What to say: *"The bilingual HTML uses Base64 encoding to safely transport Hindi Unicode through JavaScript string interpolation. The browser's `atob()` plus a small UTF-8 decoder reconstructs the original markdown, which `marked.js` then renders side-by-side."*

PART 4 — Anticipated Professor Questions (Prepare These!)

Q1: "Why didn't you use one giant agent instead of four?"

"A single agent with twenty tools confuses the LLM — it spends more tokens deciding which tool to use than actually using them. By splitting domain expertise across four agents, each agent has only 2–5 tools to choose from, which dramatically improves tool selection accuracy. It also lets me use a cheaper model for the simpler agents."

Q2: "What if OpenAI is down?"

"Two layers of fallback. First, `langchain-google-genai` is in my requirements.txt so I can swap the LLM provider in one line. Second, the search APIs are redundant — SerpAPI and Tavily, plus DuckDuckGo as a free fallback. The only single point of failure I would address next is moving from Streamlit session state to a Postgres or MongoDB persistence layer."

Q3: "Aren't LLMs known to hallucinate? How do you know the audit is correct?"

"That's exactly why I added the human-in-the-loop step. The agents produce a draft audit, but a human must click 'Approve' before the bilingual export is generated. The agents accelerate review — they don't replace human judgment. Also, every claim in the report is anchored to a source URL, which the human reviewer can verify."

Q4: "Why temperature 0.2 and not 0?"

"Temperature 0 makes the model deterministic but slightly worse at handling edge cases. 0.2 keeps it nearly factual while allowing small variations that help when input is messy. For translation I bumped it to 0.3 because translation benefits from a touch more naturalness."

Q5: "What is `recursion_limit=25`?"

"In a ReAct loop, the agent can keep calling tools indefinitely. The `recursion_limit` caps the total number of reasoning cycles. I set Agent 1 to 25 because the 3-way comparison can require several search-and-read cycles, but I capped Agents 2–4 at 15–20 to control costs. Without this limit, a runaway agent could rack up huge OpenAI bills."

Q6: "Why do you store the PDF in a temp file instead of memory?"

"PyMuPDF and pdf2image both expect a file path, not a bytes buffer. `tempfile.NamedTemporaryFile(delete=False)` gives me a real OS file that the libraries can open, and I `os.unlink()` it after extraction so nothing is left behind."

Q7: "What is the JSON knowledge base structure?"

"Each rule is stored as a JSON object with four fields: `title`, `summary`, `required_fields` (a list), and `category`. The `load_forest_rules` function globs every `.json` file in the `knowledge_base/` directory, flattens them into one list, and formats them into a numbered text block that gets injected into the LLM prompt. This way, adding a new rule means dropping a new JSON file — no code changes."

Q8: "What are the limitations of this project?"

Be honest. Pretending it's perfect makes you look naive.

"Three honest limitations. First, the knowledge base is currently small and Uttarakhand-specific — scaling to other states requires building out the JSON rules. Second, web search results can be stale or behind paywalls. Third, this is a Streamlit single-user prototype; for production I would move to FastAPI + a job queue so multiple audits can run in parallel without blocking each other."

Q9: "Can you walk me through what happens when I click 'Run Comprehensive Audit'?"

This is the **execution trace** question. Practice this:

"The button click triggers the `if start_audit:` block. It validates the form, saves the uploaded PDF to a temp file, builds the `master_project` dictionary, and loads all four cached agents. Then for each agent sequentially: it opens a `st.status` container, calls the agent's process method passing the container as `st_callback`, and the agent's streaming loop writes its thoughts and tool results live into that container. When all four finish, the reports are saved to `st.session_state.reports` so they survive subsequent reruns, and the UI switches to showing the tabbed review interface."

Q10: "How much does one full audit cost in OpenAI tokens?"

"Empirically, around 60,000–150,000 tokens total across all four agents — depending on DPR size and how many search results each agent reads. At GPT-4o pricing, that's roughly between \$0.30 and \$0.80 per full audit, plus a few cents for SerpAPI and Tavily."

(You can adjust the numbers — the point is that a confident *order-of-magnitude* answer beats "I don't know.")

PART 5 — Confidence Tactics

When you don't know something

Don't lie. Say:

"I haven't tested that specific edge case, but my hypothesis would be [X]. I would verify by [Y]."

This shows scientific thinking instead of pretending.

When the professor disagrees with a design choice

Don't argue. Say:

"That's a fair point — the trade-off I made was [reason], but in hindsight [alternative] would have been cleaner."

This shows humility and learning.

When asked something deeply technical you genuinely don't know

"Honestly sir, I'd have to read up on that. Could you point me to a reference?"

Most experienced professors **respect** this answer far more than a bluff.

PART 6 — Quick Glossary (cheat sheet)

Term	Meaning
DPR	Detailed Project Report — the big PDF infrastructure projects submit
EDS	Essential Details Sought — the rejection letter the Forest Department sends
NPV	Net Present Value — what the project pays to compensate for forest loss
RBM	River Bed Material — sand and gravel from riverbeds
MoEFCC	Ministry of Environment, Forest and Climate Change
MoRTH	Ministry of Road Transport and Highways
IRC	Indian Roads Congress (engineering code body)
IS	Indian Standard (Bureau of Indian Standards code)
PWD	Public Works Department
NOC	No Objection Certificate
KML	Google Earth file format used for geo-tagging project boundaries
OFC	Optical Fiber Cable
ReAct	Reasoning + Acting agent pattern
RAG	Retrieval-Augmented Generation
TPM	Tokens Per Minute (API rate limit)
OCR	Optical Character Recognition (image → text)

Final Words

The professor isn't trying to fail you. They're trying to see if **you actually built this** or if you just glued someone else's code together. The way to win is:

1. Speak slowly. Don't rush.
2. Use the right keywords (LangGraph, ReAct, RAG, dual-search).
3. When asked "*why*", give the trade-off — not just the choice.
4. Admit limitations honestly.
5. Show that you thought about **edge cases** (the scanned PDF fallback is your best example).

You've got this. Read this guide twice, run the application once more so the live telemetry is fresh in your memory, and walk in confident.

Good luck. 🚀